CÓDIGOCLARO · RUTA PYTHON

Tema 10 · Excepciones

Guía de estudio · DAM/DAW

try, except, else, finally y excepciones propias.

1. Explicación

Una excepción señala que una operación no pudo continuar normalmente. try delimita el código susceptible de fallar; except nombra el tipo que se sabe manejar; else se ejecuta si no hubo excepción y finally se ejecuta al salir del bloque. Capturar Exception a ciegas puede ocultar errores de programación. Para entradas convertidas a int, ValueError es un caso concreto; para un error de dominio se puede definir una excepción propia.

2. Ejemplo

Código · python

```
def convertir_entero(texto):
    try:
        return int(texto)
    except ValueError:
        return None

print(convertir_entero("12"), convertir_entero("doce"))
```

Resultado esperado

```
12 None
```

3. Errores frecuentes

- Capturar todas las excepciones y continuar con datos incompletos.
- Usar excepciones para reemplazar una condición normal que puede expresarse de forma clara.
- Escribir un except que no registra, transforma ni comunica el fallo.

4. Ejercicio

Convierte un texto en entero positivo. Devuelve None si no se puede convertir o si el número es cero o negativo.

Casos que debe superar

Entrada	Resultado esperado
"5"	5
"0"	None
"x"	None

5. Solución explicada

Código · python

```
def entero_positivo(texto):
    try:
        numero = int(texto)
    except ValueError:
        return None
    return numero if numero > 0 else None
```

Criterio de aceptación: Se acepta una excepción de dominio o una validación previa si los casos inválidos no se convierten en resultados válidos.

Otras soluciones válidas

- Capturar ValueError alrededor de int y devolver None.
- Validar el formato antes de convertir y tratar como inválido cualquier texto no admitido.

6. Fuentes oficiales

Python Software Foundation — Errors and Exceptions · Python Software Foundation — Built-in Exceptions